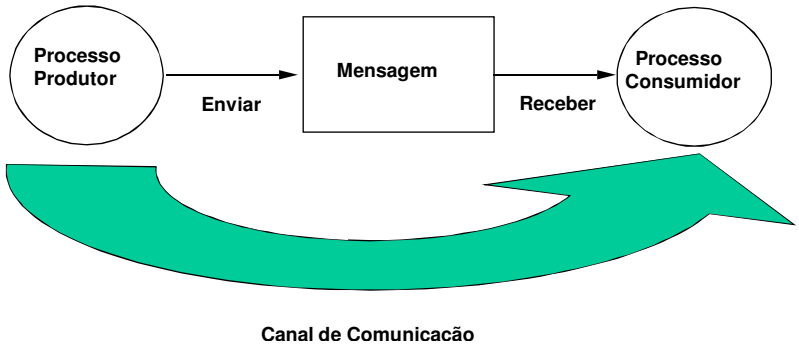
# 3-Comunicação entre Processos



# Tópicos

- Modelo de Comunicação
- Características do Canal de Comunicação
- Implementação do Canal de Comunicação
- Comunicação entre processos em UNIX (*pipes*, *named pipes*, memória partilhada)

# Comunicação entre Processos




- A comunicação entre processos pode realizar-se no âmbito:
  - de uma única aplicação,
  - uma máquina
  - Entre máquinas interligadas por uma redes de dados



# Características do Canal

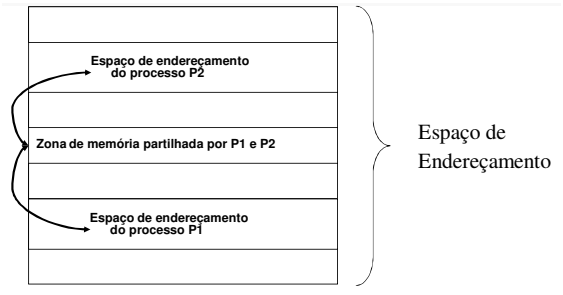
- Nomes dos objectos de comunicação
- Tipo de ligação entre o emissor e o receptor
- Estrutura das mensagens
- Capacidade de armazenamento
- Sincronização
  - no envio
  - na recepção
- Segurança-protecção envio/recepção
- Fiabilidade

# Implementação do Canal

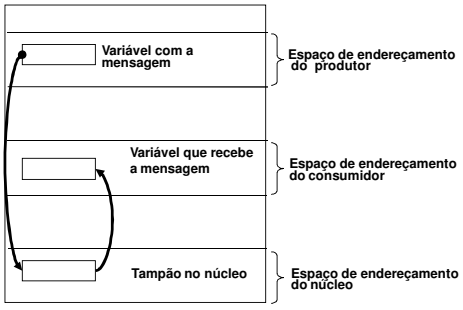
- Para implementar o canal de comunicação é fundamental definir como são transferidos os dados entre os espaços de endereçamento dos processos
- O canal de comunicação pode ser implementado com dois tipos de mecanismos:
  - **Memória partilhada**: os processos acedem a uma zona de memória que faz parte do espaço de endereçamento dos processos comunicantes
  - Transferidos através do **núcleo do sistema operativo**; os dados são sempre copiados para o núcleo antes de serem transferidos



# Implementação do Canal




Memória Partilhada




Com cópia para o Núcleo

- A memória partilhada pode ser considerada como uma extensão à gestão da memória virtual, permitindo ultrapassar os mecanismos de protecção dos espaços de endereçamento
- A comunicação no núcleo é semelhante a gestão de outros objectos do sistema em particular os ficheiros

# Implementação do Canal

- Memória partilhada:
  - mecanismo mais eficiente
  - a sincronização tem de ser explicitamente programada
  - programação complexa
- Objecto de Comunicação Sistema:
  - velocidade de transferência limitada pelas duas cópias da informação e pelo uso das chamadas sistema para Enviar e Receber
  - sincronização implícita
  - mais fácil de utilizar



# Comunicação entre processos em UNIX

- No UNIX houve uma tentativa de uniformização da interface de comunicação entre processos com a interface dos sistemas de ficheiros.
- Para perceber os mecanismos de comunicação é fundamental conhecer bem a interface com o sistema de ficheiros.

# Sistema de Ficheiros

- Sistema de ficheiros hierarquizado
- Tipos de ficheiros:
  - **Normais** - sequência de octetos (bytes) sem uma organização em registos (*records*)
  - **Ficheiros especiais** - periféricos de E/S, pipes, fifos, sockets
  - **Ficheiros directório**
- Quando um processo se começa a executar o sistema abre três ficheiros especiais
  - ***stdin*** - Input para o processo (fd - 0)
  - ***stdout***- Output para o processo (fd - 1)
  - ***stderr*** - periférico para assinalar os erros (fd - 2)
- Um descritor de ficheiro é um inteiro usado para identificar um ficheiro aberto ( os valores variam de zero até máximo dependente do sistema)



# Sistema de Ficheiros

```
int main (int argc, char *argv[])
{
  int source, destination, n;
  char buffer[1024];

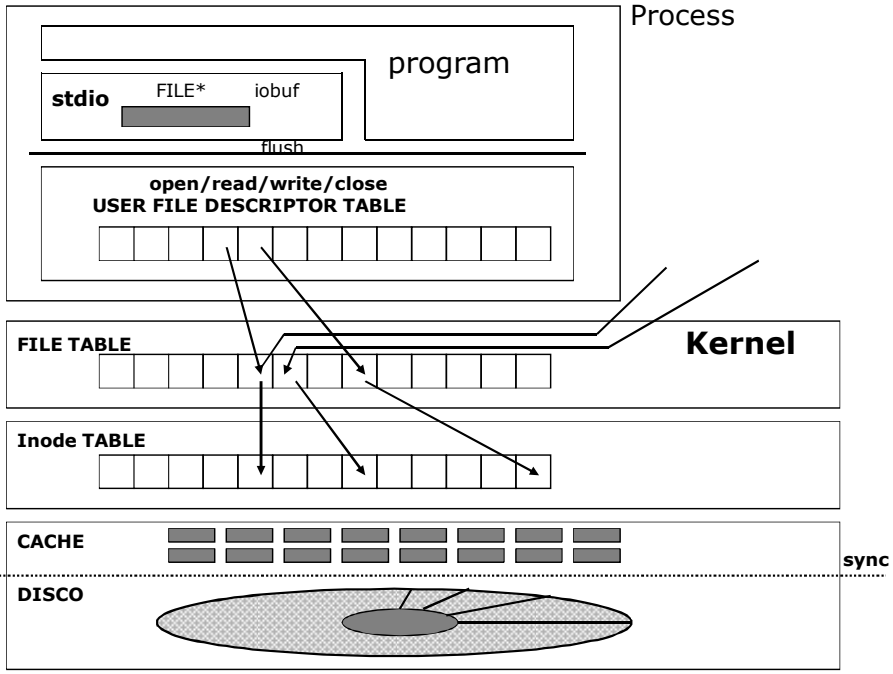
  source = open (argv[1], O_RDONLY);
  if (source == -1) {
   perror ("Opening Source File");
   exit(1);
  }

  destination = open(argv[2], O_CREAT | O_TRUNC | O_WRONLY, 0666);
  if (destination == -1) {
        perror ("Opening Destination File");
   exit(1);
  }

  while ((n = read (source, buffer, sizeof(buffer))) > 0)
   write (destination, buffer, n);

  exit(0);
}
```

# Sistema de Ficheiros






# IPC no UNIX

- *pipes*
- *named pipes* (FIFOS)
- *UNIX domain sockets*
- *Internet sockets*
- Regiões de memória partilhada

# Pipes

- Mecanismo original do UNIX para para comunicação entre processos.
- Têm uma interface idêntica à dos ficheiros
- Canal (*byte stream*) ligando dois processos
- Permite um fluxo de informação unidireccional, um processo escreve num *pipe* e o correspondente lê na outra extremidade – modelo um para um
- Não tem nome lógico associado
- As mensagens são sequências de *bytes* de qualquer dimensão



# Pipes

```
int pipe (int *fds);

   fds[0] – descritor aberto para leitura
   fds[1] – descritor aberto para escrita
```

- Os descritores de um *pipe* são análogos ao dos ficheiros
- As operações de *read* e *write* sobre ficheiros são válidas para os *pipes*
- Os descritores são locais a um processo podem ser transmitidos para os processos filhos através do mecanismo de herança
- O processo fica bloqueado quando escreve num *pipe* cheio
- O processo fica bloqueado quando lê de um *pipe* vazio

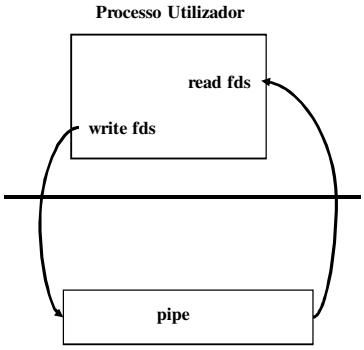
# Pipes

```
char msg[] = "utilizacao de pipes";

main() {
    char tampao[1024];
    int fds[2];

    pipe(fds);

    for (;;) {
       write (fds[1], msg, sizeof (msg));
       read (fds[0], tampao, sizeof (msg)));
    }
}
```






# Pipes

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>

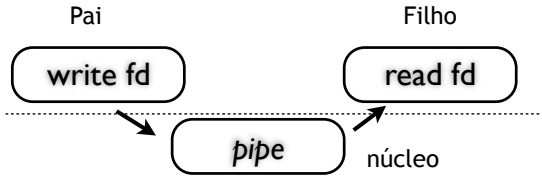
#define TAMSG 100

char msg[] = "mensagem de teste";
char tmp[TAMSG];

main() {
   int fds[2], pid_filho;

   if (pipe (fds) < 0) exit(-1);

   if (fork () == 0) { /* lê do pipe */
    read (fds[0], tmp, sizeof (msg));
    printf ("%s\n", tmp);
    exit (0);
   }
   else {
    write (fds[1], msg, sizeof (msg));
    wait(NULL);
   }
}
```



# Redirecção de Entradas/saídas

```
DUP(2)                    BSD System Calls Manual                    DUP(2)

NAME
     dup, dup2 -- duplicate an existing file descriptor

SYNOPSIS
     #include <unistd.h>

     int
     dup(int fildes);

     int
     dup2(int fildes, int fildes2);

DESCRIPTION
     Dup() duplicates an existing object descriptor and returns its value to
     the calling process (fildes2 = dup(fildes)).  The argument fildes is a
     small non-negative integer index in the per-process descriptor table.
     The value must be less than the size of the table, which is returned by
     getdtablesize(2).  The new descriptor returned by the call is the lowest
     numbered descriptor currently not in use by the process.
```



# Redirecção de Entradas/saídas

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>

#define TAMSG 100

char msg[] = "mensagem de teste";
char tmp[TAMSG];

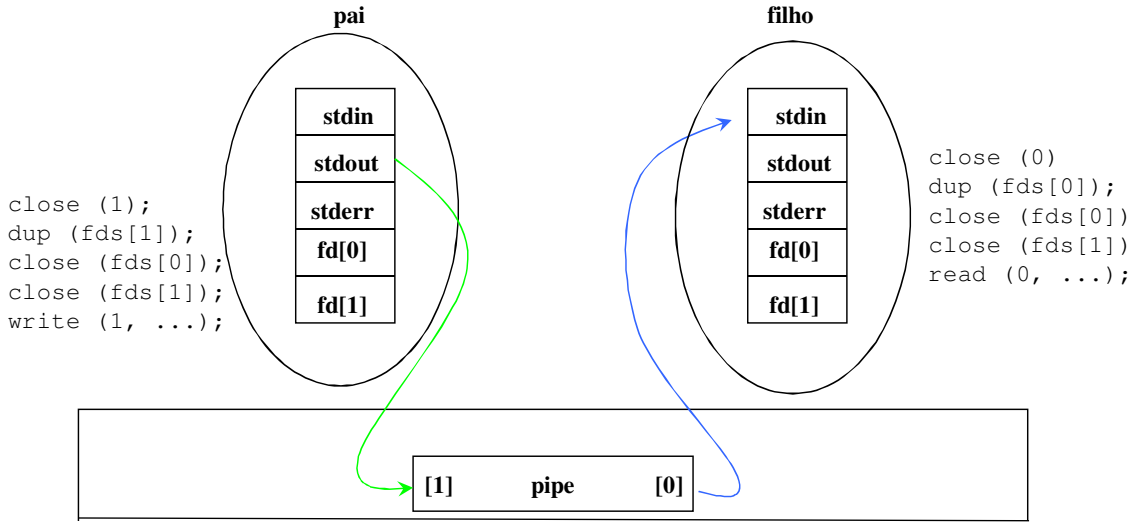
main() {
   int fds[2], pid_filho;

   if (pipe (fds) < 0) exit(-1);

   if (fork () == 0) {
    /* liberta o stdin (posição zero) */
    close(0);
    /* redirecciona o stdin para o pipe de leitura */
    dup (fds[0]);
    /* fecha os descritores não usados pelo filho */
    close (fds[0]);
    close (fds[1]);

    /* lê do pipe */
    read (0, tmp, sizeof (msg));
    printf ("%s\n", tmp);
    exit (0);
   }
   else {
       /* liberta o stdout (posição um) */
       close(1);
       /* redirecciona o stdout para o pipe de escrita */
       dup (fds[1]);
       /* fecha os descritores não usados pelo pai */
       close (fds[0]);
       close (fds[1]);
       write (1, msg, sizeof (msg));
       wait(NULL);
      }
}
```

# Redirecção de Entradas/saídas






# popen

```
POPEN(3)                BSD Library Functions Manual                POPEN(3)

NAME
     pclose, popen -- process I/O

LIBRARY
     Standard C Library (libc, -lc)

SYNOPSIS
     #include <stdio.h>

     FILE *
     popen(const char *command, const char *mode);

     int
     pclose(FILE *stream);

DESCRIPTION
     The popen() function ``opens'' a process by creating a bidirectional pipe, forking, and invoking
     the shell.  Any streams opened by previous popen() calls in the parent process are closed in the
     new child process.  Historically, popen() was implemented with a unidirectional pipe; hence, many
     implementations of popen() only allow the mode argument to specify reading or writing, not both.
     Because popen() is now implemented using a bidirectional pipe, the mode argument may request a
     bidirectional data flow.  The mode argument is a pointer to a null-terminated string which must be
     `r' for reading, `w' for writing, or `r+' for reading and writing.

     The command argument is a pointer to a null-terminated string containing a shell command line.
     This command is passed to /bin/sh, using the -c flag; interpretation, if any, is performed by the
     shell.
```

# popen

```c
//EXAMPLE
//The following program will print on the standard output the
//names of files in the current directory with a .c suffix.

#include <stdio.h>
#include <stdlib.h>

main() {
   char *cmd = "/bin/ls *.c";

   char buf[BUFSIZ]; FILE *ptr;

   if ((ptr = popen(cmd, "r")) != NULL)

    while (fgets(buf, BUFSIZ, ptr) != NULL) printf("%s", buf);

   return 0;
}
```



# *Named pipes* (FIFOS)

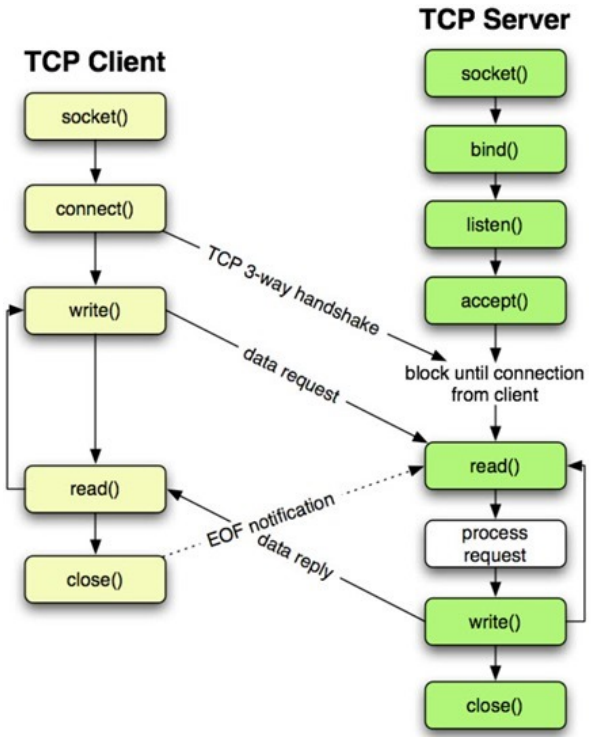
- Para dois processos(que não sejam pai e filho) comunicarem é preciso que o *pipe* seja identificado por um nome
- Atribui-se um nome lógico ao *pipe*. O espaço de nomes usado é o do sistema de ficheiros
- Um *named pipe* comporta-se externamente como um ficheiro, existindo uma entrada na directoria correspondente
- Um *named pipe* pode ser aberto por processos que não têm qualquer relação hierárquica
- Ver exemplo (fifo-srv.c, fifo-clt.c)

# *Internet Sockets*

- Um *Internet socket* ou *network socket* é um mecanismo de comunicação entre processos que podem estar em execução em diferentes máquinas ligadas por uma rede de dados
- A comunicação é bidireccional e suporta diferentes tipos de semântica de comunicação (SOCK_STREAM, SOCK_DGRAM e SOCK_RAW)
- A API é apresentada na figura seguinte.
- Ver exemplo is-server.c e is-client.c



# *Internet Sockets*



# *Unix domain sockets*

- Um UNIX domain socket ou IPC socket é um mecanismo de comunicação entre processos em execução no mesmo sistema operativo
- A comunicação é bidireccional e suporta diferentes tipos de semântica de comunicação (SOCK_STREAM, SOCK_DGRAM e SOCK_RAW)
- A API é muito semelhante à dos *Internet Sockets*, no entanto, neste caso a pilha protocolar TCP/IP não é usada.
- ver exemplo uds-server.c e uds-client.c



# Memória Partilhada

- Permite o acesso de vários processos a uma zona de memória comum
- A dimensão do segmento não pode ser alterada depois da criação
- Cada processo pode "ver" o segmento em endereços distintos do seu espaço de endereçamento
- Criação de uma região:

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
int shmget (key_t key, int size, int shmflg)
```

- `size` especifica a dimensão da região em bytes

# Memória Partilhada

- Associação a uma região:
  - `char* shmat (int shmid, char *shmaddr, int shmflg)`
- Devolve o endereço base da região
- O endereço pode:
  - ser especificado por `shmaddr`
  - se `shmaddr` for zero,o endereço é calculado pelo sistema
- Se `shmflg = SHM_RDONLY` o acesso fica restrito a leitura
- eliminação da associação:
  - `int shmdt (char *shmaddr);`
- Ver Exemplo (shmem-clt.c shmemsrv.c)